

Del 1 - Skillnaden mellan feltyper

Syntaxfel: Ett syntaxfel är ett stavfel eller regelbrott i koden ([t.ex.](#) saknar parentes, stavfel i "print"). Eftersom Python inte förstår koden stannar programmet direkt och vägrar att starta.

exempel: `print("Hej" #koden saknar slutparentes.`

Logiskt fel: Ett logiskt fel innebär att koden faktiskt körs utan att krascha, men resultatet blir fel. Eftersom tanken eller beräkningen bakom koden är felaktig.

exempel:

```
medelvärde = 10+20/2
```

```
print(medelvärde)
```

#output= 20 istället för 15 eftersom det saknas parentes runt 10+20.

Runtime-fel: Fel som uppstår medan programmet körs, oftast på grund av ogiltig data eller operationer som inte går att utföra.

exempel:

```
tal = 0
```

```
resultatet = 50/tal
```

```
print(resultatet)
```

#koden kraschar eftersom tal kan inte delas med noll.

Del 2 - Undantagshantering med try/except (kod + förklaring)

```
try:
    tal = int(input("Ange ett tal: "))
    resultat = 100 / tal
    print("100 /" , tal, "=", resultat)
except ZeroDivisionError:
    print("Fel: Kan inte dela med noll!")
except ValueError:
    print("Fel: Skriv ett tal, inte text!")
```

Förklaring:

Hur try/except fungerar: Programmet provar först att köra koden under **try**. Om något går fel stoppas inte hela programmet, utan det hoppar istället direkt till **except** och kör koden där och visar felmeddelandet.

Varför det är användbart: Det gör att programmet slipper krascha om användaren gör ett misstag. Istället kan man visa ett tydligt och vänligt felmeddelande.

Del 3 - Pseudokod

START

SKRIV UT "Ange din ålder:"

LÄS ålder

OM ålder är inte ett giltigt tal

SKRIV UT "Fel: Du måste ange en siffra"

ANNARS OM ålder ≥ 18 :

SKRIV UT "Du är vuxen"

ANNARS OM ålder ≥ 13 OCH ≤ 17 :

SKRIV UT "Du är tonåring"

ANNARS:

SKRIV UT "DU är barn"

SLUT

Del 4 - Reflektion

1. Pseudokod gör att man kan fokusera helt på hur programmet ska tänka, utan att behöva bry sig om syntax eller koda rätt direkt. Det blir som en enkel ritning för koden.
2. Man upptäcker missar och fel i sin idé tidigt. Det är mycket enklare att ändra lite vanlig text i sin planering än att behöva leta fel och skriva om färdig kod efteråt.
3. Det är viktigt att veta vilken typ av fel man letar efter (syntax, runtime eller logiskt fel). Att använda `try/except` stoppar krascher, och lite planering i början sparar massor av tid i slutändan.